

Store: state
8914

1200

clone

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
},
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "99566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  },
  "phone": "010-692-6593 x89125",
  "website": "anastasia.net",
  "company": {
    "name": "Deckow-Crist",
    "catchPhrase": "Proactive didactic contingency",
    "bs": "synergize scalable supply-chains"
  }
},
{
  "id": 3,
```

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
},
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "99566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  },
  "phone": "010-692-6593 x89125",
  "website": "anastasia.net",
  "company": {
    "name": "Deckow-Crist",
    "catchPhrase": "Proactive didactic contingency",
    "bs": "synergize scalable supply-chains"
  }
},
{
  "id": 3,
```

```
const initialState = [];
```

```
const taskSlice = createSlice({
  name: 'tasks',
  initialState,
  reducers: {
    addTask: (state, action) => {
      state.push[{id: new Date(), task: action.payload, completed: false}]
    },
    toggleTask: (state, action) => {
      state.map(task => task.id === action.payload ?
        {...task, completed: ! task.completed}: task)
    },
    clearTasks: (state) => {
      state = [];
    },
  },
})
```

actions

dispatch(addTask("Learn ReactJS"));

dispatch(clearTasks())

reducer fn

```
const initialState = [];
```

```
const taskSlice = createSlice({
  name: 'tasks',
  initialState,
  reducers: {
    addTask: (state, action) => {
      state.push[{id: new Date(), task: action.payload, completed: false}]
    },
    toggleTask: (state, action) => {
      state.map(task => task.id === action.payload ?
        {...task, completed: ! task.completed}: task)
    },
    clearTasks: (state) => {
      state = [];
    },
  },
})
```

Clone

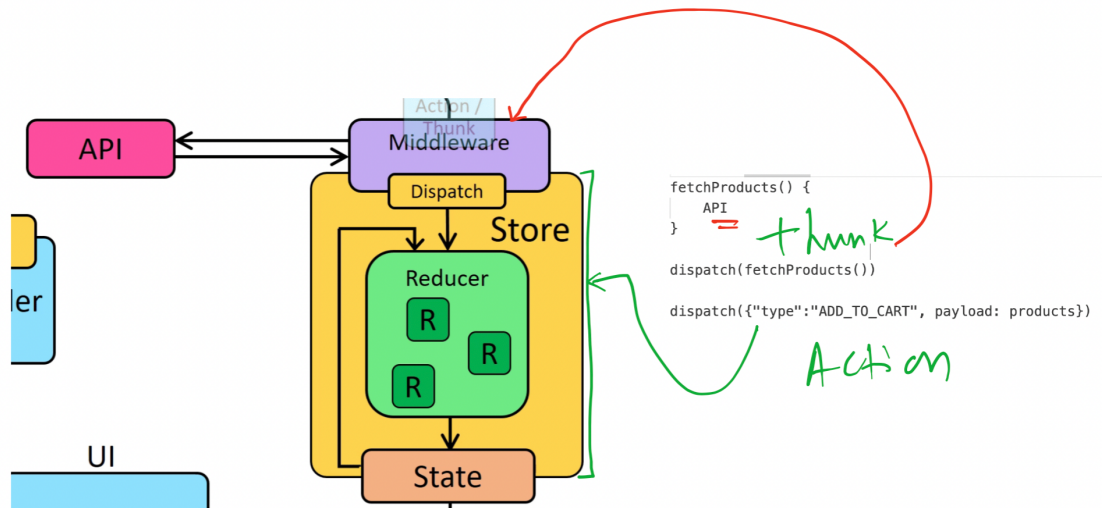
```
{
  tasks: [],
  profile: {
    name: 'John Doe',
    age: 30
  }
}
```

Store: State

```
{
  tasks: [],
  profile: {
    name: 'John Doe',
    age: 30
  }
}
```

```
dispatch(addTask("Learn ReactJS"));
```

```
dispatch(clearTasks())
```

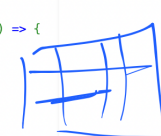


```
// First, create the thunk
export const fetchTodos = createAsyncThunk(
  'todos/fetch',
  async () => {
    const response = await fetch("https://jsonplaceholder.typicode.com/todos");
    const data = await response.json();
    return data;
  },
)
```

dispatch(fetchTodos())

Thunk

```
const todoSlice = createSlice({
  name: "todo",
  initialState,
  reducers: {
  },
  extraReducers: (builder) => {
    builder.addCase(fetchTodos.pending, (state) => {
      state.status = "loading";
      state.error = null;
      state.list = []
    });
    builder.addCase(fetchTodos.fulfilled, (state, action: PayloadAction<Todo[]>) => {
      state.status = "idle";
      state.error = null;
      state.list.push(...action.payload)
    });
    builder.addCase(fetchTodos.rejected, (state) => {
      state.status = "idle";
      state.error = "Error fetching todos!!!";
      state.list = []
    });
  }
});
```



①

②

Error